UNIVERSITÀ POLITECNICA DELLE MARCHE

Facoltà di Ingegneria

Corso di Manutenzione Preventiva per l'Automazione e la Robotica

PHM North America 2024

Professore:

Prof. Alessandro Freddi

Studenti:

Emanuele Biccheri

Emanuele Fares

Alessio Giacconi

Anno Accademico 2024/2025

Indice

Introduzione **2**

1 ETL e Analisi Esplorativa **4**

1.1 Caricamento del Dataset 4

1.2 Feature Engineering 6

1.3 Analisi Esplorativa 7

1.3.1 Violin Plot 8

1.3.2 Heat Map 11

1.4 Bilanciamento del Dataset 13

2 Regressione e PDF **15**

2.1 Metodologia 15

2.2 Selezione delle Feature 18

2.3 Addestramento dei Modelli 19

2.4 Risultati e Calcolo del Torque Margin 21

2.4.1 Performance dei modelli 23

2.4.2 Calcolo del Torque Margin 25

2.5 Probability Density Function 26

3 Classificazione **29**

3.1 Selezione delle Feature 29

3.2 Metodologia 31

3.3 Addestramento dei Modelli 34

3.4 Risultati della Classificazione 37

3.4.1 Scoring 39

3.4.2 Performance dei Modelli 41

Indice delle figure

1.1 Violin plot per `delta_T_relative`. 9
1.2 Violin plot per `np/ng ratio`. 9
1.3 Violin plot per `mgt`. 10
1.4 Violin plot per `ng`. 10
1.5 Violin plot per `np`. 10
1.6 Violin plot per `ias`. 10
1.7 Violin plot per `oat`. 10
1.8 Violin plot per `omega`. 10
1.9 Violin plot per `pa`. 11
1.10 Violin plot per `trq_margin`. 11
1.11 Violin plot per `trq_measured`. 11
1.12 Violin plot per `trq_target`. 11
1.13 Correlation Heatmap delle Feature 12

2.1 Correlation Heatmap 18
2.2 Scatter plot dei modelli di regressione. 24
2.3 PDF fittate sui residui del *Torque Margin* 27

3.1 Correlation Heatmap delle Feature 31
3.2 Grafico della funzione di Score della challenge 40
3.3 Classification Report relativo alla Classificazione Pesata 41
3.4 Matrice di Confusione relativa alla Classificazione Pesata 42
3.5 Classification Report relativo alla Classificazione con Pesi Uniformi . . 43
3.6 Matrice di Confusione relativa alla Classificazione con Pesi Uniformi . . 44

Introduzione

Nella seguente relazione verranno esposte le tecniche e metodologie di regressione e classificazione che hanno consentito di portare a termine la *PHM North America 2024 Conference Data Challenge*[1]: l'obiettivo della presente challenge è stimare lo stato di salute di motori a turbina di un elicottero, tramite tecniche di regressione e classificazione, fornendo anche un intervallo di confidenza della stima eseguita.
Il dataset fornito dagli organizzatori della challenge è costituito dai dati di 7 motori (4 per il training e 3 per il testing) e riporta le misure di alcune grandezze utili ad eseguire i task precedentemente illustrati. I dataset per training e testing forniti risultano essere molto voluminosi, in particolare il training set costituito da 742626 record e test set da 21437 record; questo fattore è stato molto importante per alcune delle scelte implementative che sono state compiute nel corso dello svolgimento della challenge.
Nella tabella 1 sono riportati nel dettaglio gli attributi contenuti nei dataset di training e testing, con il relativo significato fisico che ricoprono nel dominio applicativo in cui si sta lavorando.
Successivamente, verrà illustrata una prima fase di ETL e analisi esplorativa dei dati, volta a comprendere la struttura del dataset, le correlazioni tra le variabili e l'eventuale sbilanciamento delle classi.
Si proseguirà con l'analisi di regressione, finalizzata alla stima del Torque Margin e alla modellazione della Probability Density Function (PDF) dei residui.
Infine, sarà affrontata la fase di classificazione, con l'obiettivo di prevedere lo stato di salute del motore, fornendo anche una stima dell'intervallo di confidenza associato alla predizione.

[1] `https://data.phmsociety.org/phm2024-conference-data-challenge/`

PHM America 2024 Dataset	
Attributo	**Significato**
`Id`	Indicatore numerico del record in esame.
`Outside Air Temperature (OAT)`	Valore numerico indicante la temperatura esterna dell'aria. Influisce sulle prestazione del motore influenzando la temperatura dell'aria in ingresso.
`Mean Gas Temperature (MGT)`	Valore numerico indicante la temperatura media del gas presente all'interno del motore. Grandezza fondamentale poiché rappresenta lo stato termico interno del motore.
`Available Power (PA)`	Valore numerico che rappresenta la massima potenza che il motore può fornire nelle condizione operative attuali. Non è direttamente un indice di guasto, ma discrepanze fra potenza attesa e disponibile possono essere indice di comportamenti anomali.
`Net Power (NP)`	Valore numerico che riporta la potenza effettiva dopo aver tenuto conto delle perdite di sistema, è un indicatore dell'efficienza complessiva del motore.
`Compressor Speed (NG)`	Valore numerico che rappresenta la velocità di rotazione del compressore, che regola il flusso d'aria e la pressione all'interno del motore
`Torque Measured (trq_measured)`	Valore numerico che rappresenta la forza rotazionale prodotta dal motore che aziona il rotore principale. Parametro fondamentale per valutare le prestazioni, poiché è direttamente correlato alla capacità del motore di generare la potenza necessaria.
`Faulty`	Etichetta che classifica il record come "True", se il motore risulta essere guasto, "False", se risulta operare in condizioni normali.
`Torque Margin (trq_margin`	Valore numerico indicante la differenza tra la coppia desiderata e quella effettivamente fornita dal motore.

Tabella 1: Struttura del dataset fornito per la challenge PHM America 2024

Capitolo 1

ETL e Analisi Esplorativa

Questo capitolo descrive le fasi iniziali di preparazione del dataset, dalla fase di caricamento e integrazione dei dati fino all'analisi preliminare. I dati, forniti in formato .csv, suddivisi in set di training, test e validazione sono stati associati alla variabile target faulty, che indica lo stato di salute del motore.
Successivamente, è stata condotta un'attività di feature engineering, finalizzata alla selezione e creazione di variabili utili alla modellazione. L'analisi esplorativa ha consentito di visualizzare la distribuzione delle feature e valutarne la correlazione, passaggio fondamentale per evitare ridondanze.
Infine, è stato affrontato il problema del bilanciamento delle classi, fondamentale per garantire prestazioni affidabili nei modelli di classificazione.

1.1 Caricamento del Dataset

La prima fase del progetto ha riguardato il caricamento e la preparazione dei dati forniti nell'ambito della *PHM North America 2024 Conference Data Challenge*. Il dataset comprende misurazioni provenienti da sette motori a turbina installati su elicotteri dello stesso modello: quattro sono stati utilizzati per l'addestramento, mentre i restanti tre sono stati riservati alla fase di test. I dati sono stati anonimizzati e mescolati per impedire il riconoscimento diretto delle unità.
Il lavoro è stato svolto all'interno di un ambiente *Google Colab*, utilizzando principalmente la libreria `Pandas` per il caricamento e la gestione dei dati. I file iniziali caricati sono stati: `X_train.csv`, contenente le feature registrate dai sensori, e `y_train.csv`,

contenente la variabile target `faulty`, indicante lo stato di salute del motore (0 = sano, 1 = guasto).
Il caricamento è avvenuto direttamente da Google Drive, come illustrato nel seguente frammento di codice:

```
import pandas as pd

x_train_path =
    '/content/drive/MyDrive/PHM2024/Dataset/TrainingDataset/X_train.csv'
y_train_path =
    '/content/drive/MyDrive/PHM2024/Dataset/TrainingDataset/y_train.csv'

x_train = pd.read_csv(x_train_path)
y_train = pd.read_csv(y_train_path)
```

Una volta importati i due file, è stata verificata la coerenza tra le righe attraverso la funzione `.info()`, e successivamente si è proceduto con la fusione tramite il campo `id`, unico per ogni osservazione:

```
merged_dataset = pd.merge(x_train, y_train, on='id')
```

Questa unione ha permesso di ottenere un unico dataframe consolidato, utile per le fasi successive di analisi esplorativa e modellazione predittiva. Poiché il dataset forniva direttamente il valore di *trq_measured* e il *trq_margin* in percentuale, è stato necessario calcolare il valore atteso di coppia, denominato *trq_target*, attraverso la seguente relazione inversa:

```
merged_dataset['trq_target'] =
    merged_dataset['trq_measured'] / (1 + merged_dataset['trq_margin'] / 100)
```

Il valore *trq_target* rappresenta la coppia teorica desiderata dal motore in condizioni ideali, fondamentale per la stima dello stato di salute del sistema.
A seguito del merge e della definizione di *trq_target*, è stata effettuata un'analisi strutturale del dataframe tramite:

```
merged_dataset.info()
```

Questo controllo ha consentito di verificare l'assenza di valori nulli o anomali, accertare la dimensione e completezza del dataset, e analizzare la tipologia delle variabili (continue, categoriali, intere). L'intero processo di caricamento, unione e verifica ha costituito la base per garantire la qualità e l'affidabilità delle elaborazioni successive, assicurando che i dati fossero pronti per l'analisi esplorativa e la costruzione dei modelli predittivi.

1.2 Feature Engineering

A partire dal dataset integrato, è stata avviata una fase di feature engineering, ovvero un insieme di tecniche volte a trasformare, combinare o creare nuove variabili (feature) a partire da quelle originarie, con l'obiettivo di migliorare la rappresentazione del fenomeno studiato e facilitare la modellazione.
Una delle prime operazioni ha riguardato il calcolo del *torque_target*, già descritto in precedenza, fondamentale per determinare lo scostamento percentuale espresso dal *torque_margin*. Successivamente, sono state introdotte tre nuove feature, ottenute a partire da combinazioni di grandezze già presenti nel dataset:

- **delta_T_relative**: definita come il rapporto tra la differenza tra la temperatura media dei gas e la temperatura dell'aria esterna, rispetto alla temperatura dei gas:

$$\texttt{delta_T_relative} = \frac{MGT - OAT}{MGT} \tag{1.1}$$

  Questa feature derivata permette di normalizzare il gradiente termico in funzione del regime operativo del motore, fornendo una misura sintetica delle condizioni termiche.

- **omega**: definita come il rapporto tra la potenza netta e la coppia misurata:

$$\texttt{omega} = \frac{NP}{TRQ_MEASURED} \tag{1.2}$$

  Questa feature consente di valutare l'efficienza dinamica tra la potenza prodotta e la coppia generata, aggregando informazioni provenienti da due grandezze fondamentali.

- **np_ng_ratio**: calcolata come rapporto tra la velocità netta (`NP`) e quella del compressore (`NG`):

$$\texttt{np_ng_ratio} = \frac{NP}{NG} \tag{1.3}$$

  Questa variabile cattura il bilanciamento tra due componenti meccaniche in rotazione e può contribuire a descrivere il comportamento del sistema in diverse condizioni operative.

L'introduzione delle nuove feature, sfruttando la libreria `pandas`, ha permesso di arricchire la descrizione del sistema mantenendo il dataset compatto e coerente senza introdurre ridondanze.
Il risultato è un insieme di variabili più esplicative e astratte, pensate per agevolare la fase di modellazione e migliorare l'affidabilità delle analisi successive.

1.3 Analisi Esplorativa

Completata la fase di preparazione e arricchimento del dataset, si è proceduto con un'analisi esplorativa dei dati, finalizzata a comprenderne la distribuzione, le relazioni tra le variabili e le eventuali differenze tra motori sani e guasti.
Questa fase ha rappresentato un passaggio chiave per validare la qualità dei dati e verificare alcune intuizioni fisiche legate al funzionamento del motore a turbina. In particolare, è stato utile osservare l'andamento di variabili come temperatura, velocità, potenza e coppia in funzione dello stato di salute del sistema.
Per l'analisi, sono stati utilizzati strumenti di visualizzazione come i violin plot, per confrontare la distribuzione delle feature rispetto alla variabile target *faulty*, e la matrice di correlazione, per evidenziare eventuali legami lineari tra le grandezze.
L'analisi ha permesso di:

- mettere in evidenza differenze marcate tra motori sani e guasti;
- identificare anomalie, distribuzioni asimmetriche o potenziali outlier;
- evidenziare correlazioni elevate che potrebbero introdurre ridondanza.

I risultati di questa fase hanno guidato la selezione delle feature e anticipato la necessità di interventi di bilanciamento delle classi, descritti nel paragrafo 1.4.

1.3.1 Violin Plot

I violin plot rappresentano una combinazione tra un box plot e una stima della densità della distribuzione delle variabili. In questa sezione vengono analizzate visivamente alcune delle principali feature del dataset in relazione allo stato di salute del motore (`faulty = 0` per motori sani, `faulty = 1` per motori guasti).
Questo tipo di rappresentazione è stato scelto in quanto consente di cogliere simultaneamente informazioni sulla distribuzione, la simmetria, la presenza di outlier e la differenza tra classi, offrendo così un quadro intuitivo e comparativo dell'effetto che lo stato di salute ha sulle singole variabili.
Sono stati generati i seguenti grafici:

- **delta_T_relative**: mostra una leggera differenza tra le due classi, suggerendo che i motori guasti tendano ad avere un valore medio leggermente più alto (Figura 1.1).
- **np/ng ratio**: mostra una variazione sensibile, suggerendo una leggera disomogeneità nei rapporti tra le due velocità nei motori guasti (Figura 1.2).
- **mgt (Mean Gas Temperature)**: presenta distribuzioni simili tra i due gruppi, ma con leggere differenze nei quartili superiori (Figura 1.3).
- **ng (Compressor Speed)**: mostra una distribuzione più concentrata per i motori sani, mentre i guasti presentano una maggiore variabilità (Figura 1.4).
- **np (Net Power)**: simmetrico per entrambi i gruppi, ma con una densità più elevata nei valori centrali per i motori sani (Figura 1.5).
- **ias (Indicated Airspeed)**: evidenzia una leggera asimmetria, con i motori guasti distribuiti su una gamma più ampia di valori (Fig. 1.6).
- **oat (Outside Air Temperature)**: la distribuzione per i motori guasti presenta maggiore dispersione, con valori anche negativi più marcati (Figura 1.7).
- **omega**: evidenzia una differenza più marcata tra le classi, con maggiore variabilità nei motori guasti (Figura 1.8).
- **pa (Available Power)**: indica una maggiore dispersione nei motori guasti, suggerendo fluttuazioni significative nella potenza disponibile (Figura 1.9).

- **trq_margin (Torque Margin)**: mostra una separazione chiara tra i due gruppi, rendendola una delle feature più significative nel distinguere lo stato del motore (Figura 1.10).
- **trq_measured (Torque Measured)**: mostra una leggera differenza di distribuzione tra i due gruppi, con valori lievemente più elevati nei motori sani (Figura 1.11).
- **trq_target (Torque Target)**: presenta distribuzioni paragonabili a quella di `trq_measured`, mantenendo una differenza visibile tra le due classi, coerente con la definizione del margine di coppia (Figura 1.12).

Le rappresentazioni grafiche si sono rivelate utili per cogliere pattern non immediatamente evidenti tramite analisi statistiche tradizionali. In particolare, feature come *trq_margin*, *omega* e *oat* hanno evidenziato differenze più marcate tra i due gruppi, risultando quindi particolarmente promettenti per le fasi successive di classificazione.

Figura 1.1: Violin plot per `delta_T_relative`.

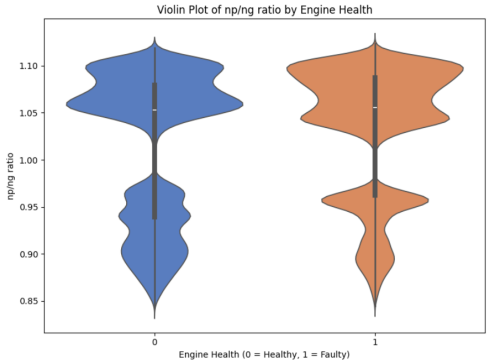

Figura 1.2: Violin plot per `np/ng ratio`.

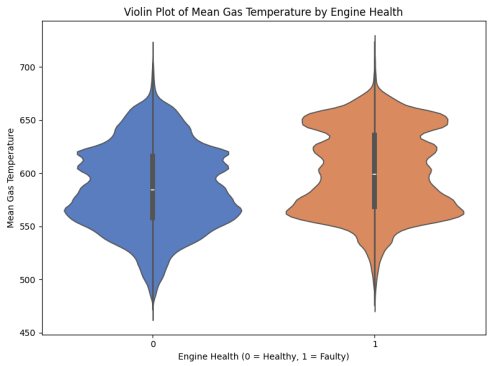

Figura 1.3: Violin plot per `mgt`.

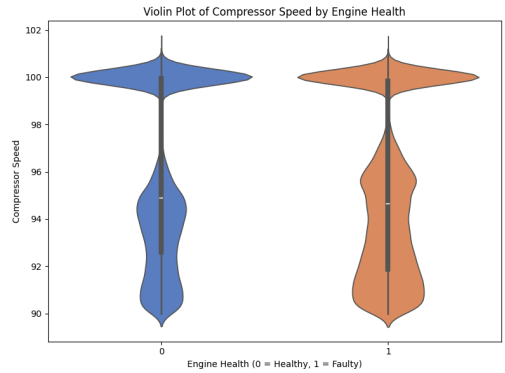

Figura 1.4: Violin plot per `ng`.

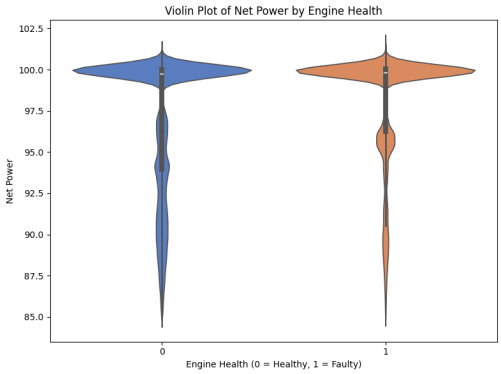

Figura 1.5: Violin plot per `np`.

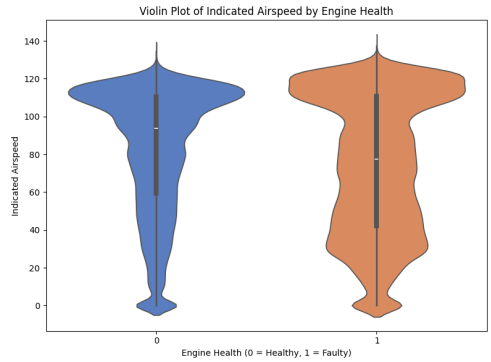

Figura 1.6: Violin plot per `ias`.

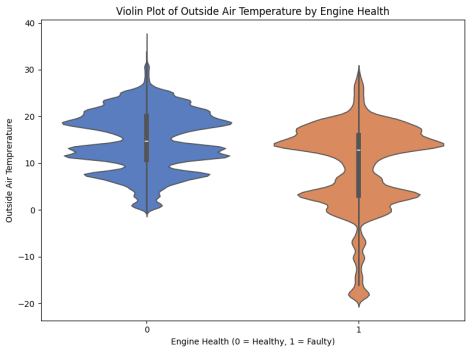

Figura 1.7: Violin plot per `oat`.

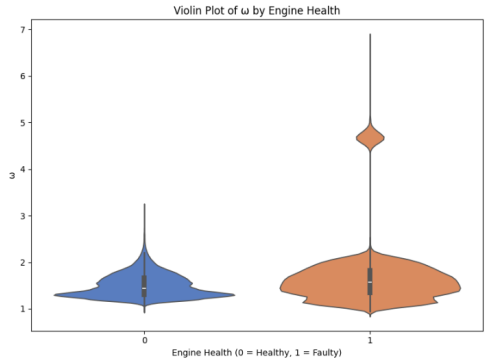

Figura 1.8: Violin plot per `omega`.

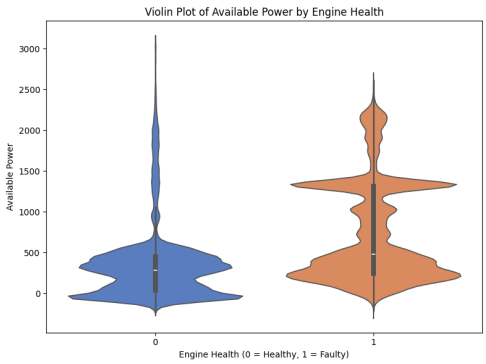

Figura 1.9: Violin plot per `pa`.

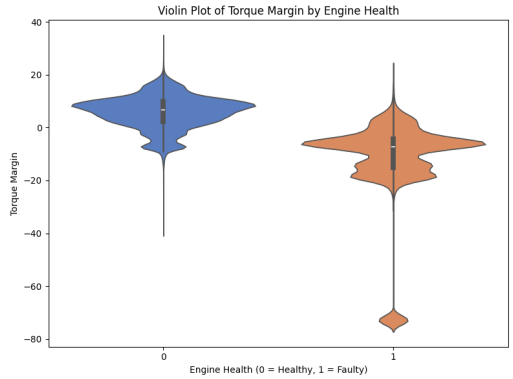

Figura 1.10: Violin plot per `trq_margin`.

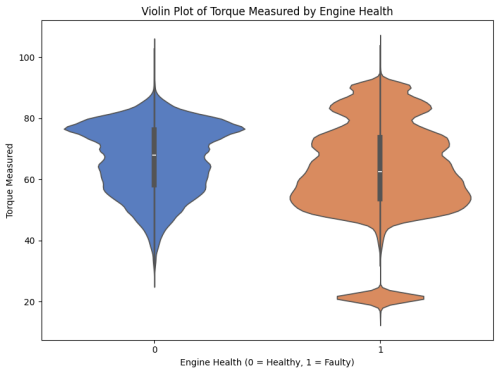

Figura 1.11: Violin plot per `trq_measured`.

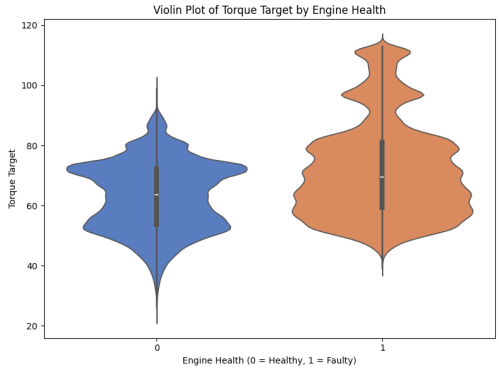

Figura 1.12: Violin plot per `trq_target`.

1.3.2 Heat Map

La Correlation Heatmap (Figura 3.1) rappresenta visivamente i coefficienti di correlazione di Pearson tra tutte le variabili numeriche del dataset, inclusi i target *trq_margin*, *trq_target* e l'etichetta binaria *faulty*, utilizzata per indicare lo stato di salute del motore (0 = sano, 1 = guasto).

Per generare la mappa, è stata prima esclusa la colonna identificativa *id* e poi riordinata la matrice di correlazione in modo da evidenziare la relazione tra la variabile *faulty* e tutte le altre grandezze. Inoltre, è stata applicata una maschera triangolare per nascondere la parte superiore della matrice e renderla più leggibile. I colori caldi (rosso) indicano correlazioni positive, mentre i colori freddi (blu) indicano correlazioni negative. Le osservazioni principali emerse dalla heatmap sono le seguenti:

- La variabile trq_margin mostra una **forte correlazione negativa con faulty**

(-0.61), suggerendo che un margine di coppia più elevato sia associato a un motore sano. Questa relazione è risultata cruciale anche per la fase successiva di classificazione.

- Le feature *trq_measured*, *ias*, *ng*, *np* e *delta_T_relative*, selezionate per la fase di regressione, presentano tutte **correlazioni deboli o moderate** con il target *trq_target*, ma nel complesso mantengono bassa collinearità reciproca, un aspetto desiderabile per la robustezza del modello.

- *ng* (Compressor Speed) e *np* (Net Power) mostrano una **correlazione negativa tra loro (-0.79)**, mentre il rapporto derivato *np_ng_ratio*, utilizzato in classificazione, presenta **valori di correlazione molto alti** con entrambe le variabili originali (*np*: +0.94, *ng*: -0.95), come atteso.

- La variabile derivata *omega* mostra correlazioni contenute sia con *faulty* (-0.32) che con le feature da cui è calcolata (*np*, *trq_measured*), ma mantiene una certa utilità informativa essendo una trasformazione di due misure chiave del motore.

- La variabile *oat* (Outside Air Temperature) risulta moderatamente correlata con *trq_target* (0.70) e inversamente correlata con *faulty* (-0.34), evidenziando un possibile legame tra condizioni esterne e malfunzionamenti.

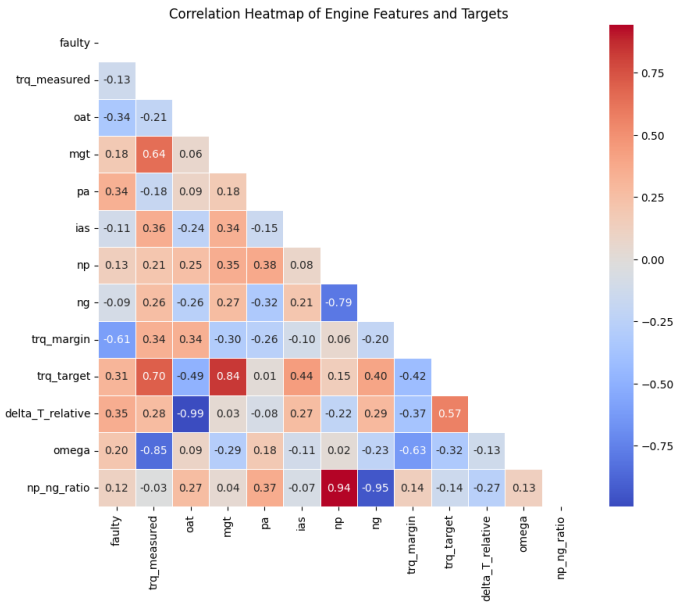

Figura 1.13: Correlation Heatmap delle Feature

Questa rappresentazione ha guidato diverse scelte nel processo di `feature selection`, sia per la regressione sia per la classificazione, evitando la selezione di variabili ridondanti o altamente collineari. È stata inoltre utile per:

- identificare feature rilevanti ma non troppo correlate tra loro, riducendo il rischio di overfitting;
- confermare il ruolo centrale del trq_margin come indicatore dello stato del motore;
- validare visivamente la coerenza delle relazioni fisiche note tra le variabili.

1.4 Bilanciamento del Dataset

Una delle sfide comuni nell'ambito dell'apprendimento supervisionato è la presenza di dataset sbilanciati, ovvero insiemi di dati in cui le classi non sono equamente rappresentate. In questo caso, la variabile target *faulty*, indicante lo stato del motore, presentava un'evidente sproporzione tra le due classi:

- **Classe 0 (motore sano)**: 443.207 istanze
- **Classe 1 (motore guasto)**: 299.418 istanze

Tale squilibrio può portare i modelli di classificazione a sviluppare un pregiudizio verso la classe maggioritaria, riducendo l'accuratezza nel rilevare i casi di guasto, che in ambito industriale rappresentano invece le condizioni più critiche da identificare.
Per affrontare il problema, è stata adottata una strategia di bilanciamento mediante upsampling, implementata con la funzione `resample` della libreria `sklearn.utils`. Nello specifico, le istanze appartenenti alla classe minoritaria (*faulty = 1*) sono state replicate casualmente con rimpiazzo (`bootstrap sampling`) fino a raggiungere lo stesso numero di elementi della classe maggioritaria.
Il nuovo dataset risultante presenta quindi una distribuzione equilibrata:

- **Classe 0 (motore sano)**: 443.207 istanze
- **Classe 1 (motore guasto)**: 443.207 istanze

Successivamente, è stata introdotta una colonna *weights*, contenente un peso associato a ciascuna istanza calcolato in modo inversamente proporzionale alla frequenza della classe nel dataset originale. In questo modo:

- Classe 0 $\rightarrow$ peso $\approx \frac{1}{443.207} \approx 2.26 \times 10^{-6}$
- Classe 1 $\rightarrow$ peso $\approx \frac{1}{299.418} \approx 3.34 \times 10^{-6}$

L'aggiunta dei pesi non incide sul bilanciamento del nuovo dataset, ma si è rivelata utile nel caso in cui si volesse addestrare un modello mantenendo memoria della distribuzione reale originaria.
Infine, prima di proseguire con la modellazione, è stato eseguito uno `shuffle` completo del dataset (tramite `sample(frac=1)`), al fine di rimuovere ogni possibile ordine sistematico che potesse introdurre bias o pattern artificiali durante l'addestramento.
Questa fase ha permesso di garantire condizioni più eque per l'apprendimento supervisionato, riducendo il rischio di overfitting sulla classe dominante e migliorando la capacità dei modelli di identificare correttamente i guasti, obiettivo primario nel contesto della manutenzione predittiva.

Capitolo 2

Regressione e PDF

In questo capitolo saranno presentate le operazioni eseguite per il task di regressione e per il calcolo della Probability Density Function (PDF). Infatti, come anticipato nel Capitolo 1, prima di passare alla fase di classificazione per la classe *faulty*, procediamo con lo stimare il valore del *torque_margin* da utilizzare in seguito.
Sarà inizialmente affrontata la scelta delle features selezionate per addestrare i modelli di regressione, ponendo particolare attenzione alle correlazioni reciproche. Successivamente, sarà presentata la metodologia che è stata applicata per raggiungere lo scopo della stima del *torque_target* utile al calcolo del *torque_margin*, per poi entrare nel dettaglio dei modelli utilizzati e dei relativi parametri. Infine, ci si concentrerà sulla valutazione dei risultati della regressione e sulla scelta della PDF più adatta.

2.1 Metodologia

Dopo aver analizzato i requisiti e gli obiettivi della challenge, si è deciso di procedere con il task di regressione prima di quello di classificazione. L'obiettivo principale della regressione è stimare il Torque Target, valore di riferimento fondamentale per il funzionamento del motore, che verrà successivamente utilizzato per il calcolo del Torque Margin. Quest'ultimo, infatti, è presentato sul sito ufficiale della challenge come parametro indicatore dello stato di salute del motore.
Il Torque Margin è definito come:

$$\text{Torque Margin} = 100 \times \frac{\text{Torque Measured} - \text{Torque Target}}{\text{Torque Target}}$$

Questa formula esprime il Torque Margin in percentuale, evidenziando la differenza tra il valore misurato e quello Target, normalizzata rispetto al valore Target. Un valore elevato del margin indica che il motore opera al di sopra del valore di riferimento, mentre un Margin basso (o negativo) può indicare una condizione di stress o deterioramento.
Poiché il Torque Margin è una grandezza derivata, il primo passo è quello di stimare il Torque Target tramite un modello di regressione. La scelta di predire il Torque Target, anziché il Torque Margin direttamente, permette di fissare un riferimento stabile e ben definito, evitando la perdita di informazioni che potrebbe derivare da una stima diretta del Margin. Successivamente, il Torque Margin viene calcolato utilizzando la formula sopra esposta.
Per la stima del Torque Target, sono stati addestrati quattro modelli di regressione differenti, i quali sono stati confrontati per selezionare quello con le migliori performance predittive:

- **Linear Regression**: un modello semplice e interpretabile che assume una relazione lineare tra le feature e il torque target.
- **Random Forest**: un ensemble di alberi decisionali che permette di catturare relazioni non lineari e interazioni complesse, riducendo il rischio di overfitting grazie alla media delle predizioni.
- **XGBoost**: un algoritmo di boosting che utilizza alberi decisionali in sequenza per correggere gli errori dei modelli precedenti, spesso ottenendo elevate performance in termini di accuratezza.
- **LightGBM**: un metodo di boosting che utilizza tecniche di gradient boosting ottimizzate per la velocità e la scalabilità, particolarmente efficace su dataset di grandi dimensioni.

Dopo aver addestrato i modelli, il migliore in termini di metriche (MAE, RMSE, R^2 e MAPE) è stato utilizzato per predire il *trq_target* sul set di test. Con il valore stimato, si calcola il *trq_margin* mediante la formula sopra riportata.
Una volta calcolati i residui (la differenza tra il *trq_margin* misurato e quello calcolato), sono state testate alcune distribuzioni di probabilità per identificare quella che meglio approssima la distribuzione effettivamente ottenuta.
Le PDF vengono applicate sui residui, cioè sulla differenza tra il torque margin reale

e quello calcolato a partire dal torque target predetto. Saranno testate le seguenti distribuzioni:

- **Normale**: assume che i residui seguano una distribuzione gaussiana; è utile se gli errori sono simmetrici e concentrati attorno a zero.
- **Laplace**: caratterizzata da code più pesanti rispetto alla distribuzione normale, utile se si osservano errori occasionali di maggiore entità.
- **Cauchy**: presenta code ancora più pesanti e una varianza teorica infinita, adatta a situazioni in cui sono presenti outlier estremi.
- **Logistic**: simile alla normale ma con code leggermente più pesanti; può essere utile quando gli errori non seguono perfettamente la simmetria gaussiana.
- **Uniforme**: distribuzione a "top piatto", dove la densità è costante su un intervallo; può essere vantaggiosa se i residui sono confinati in un range stretto, garantendo un punteggio massimo finché l'errore rimane al di sotto di una certa soglia.

Per verificare quale delle PDF proposte si adatti meglio ai dati, è stato utilizzato il **Kolmogorov-Smirnov (KS) Test**, un test non parametrico che confronta la funzione di distribuzione cumulativa (CDF) empirica dei residui con quella teorica della distribuzione candidata, restituendo due valori:

- **Statistica D**: rappresenta la massima differenza assoluta tra la CDF empirica e quella teorica. Un valore di D minore indica una migliore aderenza della distribuzione ai dati.
- **p-value**: fornisce la probabilità, sotto l'ipotesi nulla (ossia che i dati seguano la distribuzione candidata), di osservare una discrepanza almeno così grande come quella calcolata. Un *p-value* basso indica che la differenza osservata è statisticamente significativa, suggerendo che la distribuzione candidata non si adatta bene ai dati.

In questo contesto, il KS Test è stato applicato per confrontare le PDF fittate ai residui, permettendo di selezionare quella che minimizza la statistica D e offre, al contempo, un *p-value* ragionevolmente elevato (sebbene in dataset di grandi dimensioni, anche piccole differenze possano portare a *p-value* molto bassi).

2.2 Selezione delle Feature

Le feature sono state selezionate valutando la loro correlazione con il parametro *trq_target*. Di seguito, in Figura 2.1, viene riportata, per comodità, la matrice di correlazione delle features.

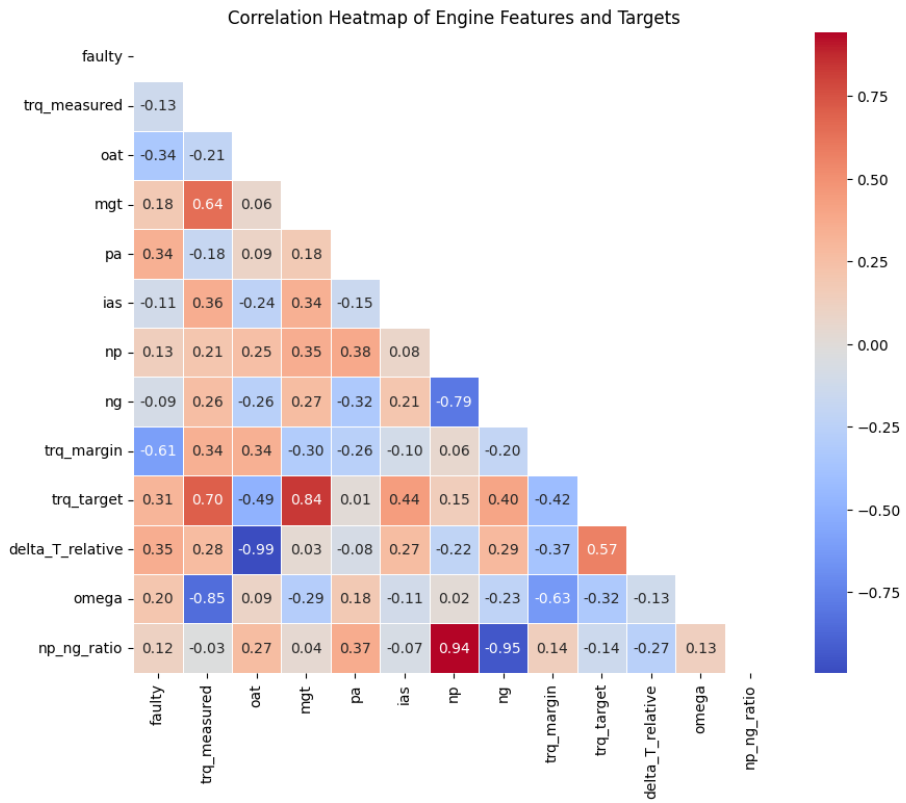

Figura 2.1: Correlation Heatmap

Dopo l'analisi della Correlation Heatmap, le feature scelte sono state:

- **trq_measured**: presenta una correlazione di 0.70, indicando che all'aumentare del *trq_target*, il *trq_measured* tende a seguire la stessa tendenza. È stata scelta perché rappresenta la misura effettiva della coppia generata dal motore. Poiché il*trq_target* definisce il valore di riferimento che il motore dovrebbe raggiungere, il confronto con il valore misurato permette di analizzare la dinamica del motore e le discrepanze tra stato ideale e stato reale.

- **ias**: mostra una correlazione positiva di 0.44, un valore intermedio che risulta significativo senza essere eccessivamente elevato. Infatti, feature con correla-

zioni troppo basse sarebbero state escluse per la loro scarsa rilevanza, mentre correlazioni troppo alte avrebbero potuto portare a problemi di overfitting.

- **ng**: ha un indice di correlazione di 0.40, simile a quello di *ias*. Per questo motivo, la selezione di questa feature è stata motivata dagli stessi criteri, in quanto contribuisce a spiegare la variabilità del *trq_target* senza introdurre ridondanze.

- **delta_T_relative**: con una correlazione di 0.57, rientra tra le feature più significative. In questo caso, è stata fatta un'ulteriore considerazione: essendo una grandezza derivata da *mgt* e *oat*, si è deciso di escludere queste ultime due per evitare ridondanze informative e il rischio di overfitting.

- **np**: presenta una correlazione più bassa, pari a 0.15. Nonostante il valore relativamente ridotto, è stata inclusa poiché rappresenta la potenza netta del motore, una variabile fondamentale per comprendere la sua effettiva capacità di generare lavoro e, quindi, un indicatore diretto delle prestazioni e dell'efficienza del sistema.

L'esclusione delle feature non selezionate, dunque, è stata fatta tenendo in considerazione indici di correlazione troppo alti o troppo bassi con il *trq_target*, oltre all'eventuale contributo fornito nel calcolare le feature derivate. Inoltre, è stato anche tenuto in considerazione l'eventuale eccessiva correlazione tra le stesse feature selezionate.

2.3 Addestramento dei Modelli

Successivamente alla scelta delle feature, si è passati all'addestramento dei modelli.
Per prima cosa, è stato suddiviso il dataset in *training set* e *test set* con proporzione di 80-20, ovvero utilizzando l'80% per l'addestramento dei modelli e il 20% per la fase di test. In seguito, è stata applicata una normalizzazione mediante il metodo *StandardScaler*, della libreria *sklearn*, in modo da portare le feature su una scala comune.

```
# Suddivisione in training e test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
        random_state=42)

# Standardizzazione delle feature
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Successivamente, sono stati definiti i modelli presentati in Sezione 2.1.

```
# Definizione dei modelli
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=50,
            max_depth=10, random_state=42),
    "XGBoost": xgb.XGBRegressor(n_estimators=50, device='cuda',
            max_depth=10, random_state=42),
    "LightGBM": lgb.LGBMRegressor(n_estimators=50, boosting_type='gbdt',
            max_depth=10, device='cpu', random_state=42)
}
```

A seguito di diverse prove e confronti, sono state scelte le seguenti configurazioni dei modelli:

- **Linear Regression**:
  - Non richiede un tuning intensivo dei parametri, fornendo così un benchmark utile per confrontare le prestazioni degli altri modelli.
- **Random Forest Regressor**:
  - *n_estimators = 50*: numero di alberi della foresta, scelto per garantire stabilità nelle predizioni, tempi di elaborazione contenuti e ridurre il rischio di overfitting.
  - *max_depth = 10*: profondità massima di ogni albero, impostata per limitare la complessità del modello e favorire la generalizzazione.
  - *random_state = 42*: impostato per assicurare la ripetibilità degli esperimenti.
- **XGBoost Regressor**:
  - *n_estimators = 50*: numero di alberi utilizzati per il boosting, mantenuto in coerenza con gli altri modelli per garantire tempi di training efficienti.
  - *device = 'cuda'*: abilita l'uso della GPU per accelerare l'addestramento sfruttando il calcolo parallelo.

  - *max_depth = 10*: limita la profondità degli alberi, contribuendo a prevenire l'overfitting e migliorare la generalizzazione.
  - *random_state = 42*: garantisce la ripetibilità dei risultati.

- **LightGBM Regressor**:
  - *n_estimators = 50*: numero di iterazioni (alberi) del modello, scelto per mantenere coerenza con gli altri metodi.
  - *boosting_type = 'gbdt'*: utilizza il Gradient Boosting Decision Tree, noto per la sua efficienza e precisione su dataset di grandi dimensioni.
  - *max_depth = 10*: imposta la profondità massima degli alberi per ridurre il rischio di overfitting e velocizzare il training.
  - *device = 'cpu'*: In questo caso, LightGBM viene eseguito sulla CPU, sfruttando le ottimizzazioni specifiche del framework.
  - *random_state = 42*: impostato per garantire la ripetibilità degli esperimenti.

Una volta definiti i modelli, si è proceduto con l'addestramento e la valutazione delle prestazioni. In particolare, è stato implementato un ciclo *for* per iterare sui quattro modelli scelti. All'interno di questo ciclo, per ciascun modello, si è invocato il metodo *fit* utilizzando il *training set* scalato e le etichette reali corrispondenti.

```
    model.fit(X_train_scaled, y_train)
```

2.4 Risultati e Calcolo del Torque Margin

Una volta concluso l'addestramento, all'interno dello stesso ciclo *for* usato per l'addestramento in Sezione 2.3, attraverso il metodo *predict*, sono state eseguite le predizioni dei modelli addestrati.

```
    y_pred = model.predict(X_test_scaled)
```

In seguito, sono state definite delle funzioni che permettono di valutare le prestazioni dei modelli mediante le seguenti metriche:

- **MAE (Mean Absolute Error):**

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

Misura l'errore medio assoluto tra i valori predetti e quelli reali, fornendo un'indicazione diretta della deviazione media senza penalizzare eccessivamente gli outlier.

- **MSE (Mean Squared Error) e RMSE (Root Mean Squared Error):**

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 , \quad \text{RMSE} = \sqrt{\text{MSE}}$$

L'MSE penalizza maggiormente gli errori più elevati poiché li eleva al quadrato, mentre l'RMSE, riportato nelle unità del target, fornisce un'idea dell'errore medio quadratico.

- **$\mathbf{R^2}$ (Coefficient of Determination):**

$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

Indica la proporzione di varianza dei dati spiegata dal modello. Valori vicini a 1 indicano un'ottima capacità predittiva, mentre valori inferiori suggeriscono una minore accuratezza.

- **MAPE (Mean Absolute Percentage Error):**

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

Espressione in percentuale del MAE, questa metrica indica l'errore medio relativo, facilitando l'interpretazione dell'accuratezza del modello in termini percentuali.

Le metriche vengono calcolate attraverso i metodi messi a disposizione dalla libreria *sklearn* nel seguente modo:

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
    r2 = r2_score(y_test, y_pred)
    mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
```

Per ogni modello, viene creato uno scatter plot che mette in relazione i valori reali (y_{test}) e quelli predetti (y_{pred}).

```
    # Visualizzazione della predizione vs valore reale
    plt.figure(figsize=(6, 4))
    plt.scatter(y_test, y_pred, alpha=0.5)
    plt.xlabel("True Values")
    plt.ylabel("Predicted Values")
    plt.title(f"{name} Predictions vs True Values")
    plt.plot([y.min(), y.max()], [y.min(), y.max()], '--r')
    plt.show()
```

Questa combinazione di analisi visiva e quantitativa offre un quadro completo delle performance dei modelli, permettendo di individuare il modello ottimale per la stima del *trq_target*, che costituisce il riferimento per il calcolo del *trq_margin*, parametro fondamentale per la diagnosi dello stato di salute del motore.

2.4.1 Performance dei modelli

Dunque, i risultati dell'addestramento sono stati valutati sia visivamente, tramite scatter plot, sia quantitativamente, mediante diverse metriche di performance.
Gli scatter plot mettono a confronto i valori predetti e quelli reali per ciascun modello. Questi grafici mostrano come i punti si distribuiscano attorno alla linea diagonale di riferimento, la quale rappresenta la condizione ideale in cui la predizione coincide esattamente con il valore reale. Dall'osservazione degli scatter plot (Figura 2.2), si nota che il modello XGBoost produce una dispersione minima e più lineare rispetto agli altri.

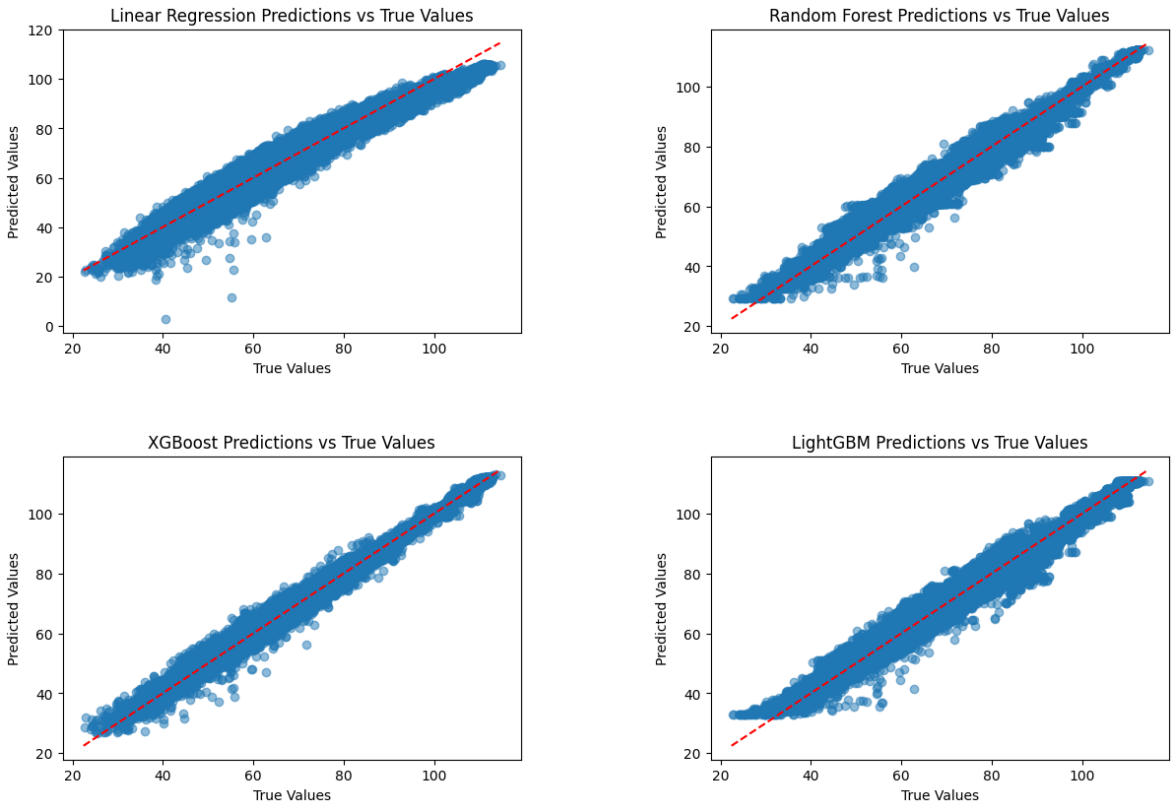

Figura 2.2: Scatter plot dei modelli di regressione.

I risultati ottenuti dai modelli di regressione sono riassunti nella Tabella 2.1.

Tabella 2.1: Confronto delle prestazioni dei modelli di regressione

Modello	**MAE**	**RMSE**	R^2	**MAPE (%)**
Linear Regression	2.709558	3.323883	0.948330	4.130743
Random Forest	1.270990	1.791347	0.984993	2.025313
XGBoost	0.740955	1.103416	0.994306	1.185288
LightGBM	1.377969	1.848608	0.984018	2.147599

Dall'analisi dei risultati, si evidenzia quanto segue:

- **Linear Regression**: Pur fornendo una stima accettabile, il modello lineare mostra un errore medio (MAE) e un RMSE più elevati, con un R^2 inferiore rispetto agli altri modelli, il che indica una minore capacità di catturare la complessità delle relazioni nei dati.

- **Random Forest** e **LightGBM**: Entrambi i modelli basati su ensemble raggiungono prestazioni simili, con valori di MAE, RMSE e MAPE che sono inferiori rispetto alla regressione lineare e con R^2 molto vicini a 1. Questo conferma la loro capacità di modellare relazioni non lineari e interazioni complesse tra le feature.

- **XGBoost**: Il modello XGBoost si distingue per le migliori prestazioni complessive, con il MAE e il RMSE più bassi, un R^2 molto vicino a 1 e il MAPE minore. Questi risultati indicano che XGBoost è in grado di catturare in maniera molto precisa le relazioni tra le feature e il torque target.

Alla luce dei risultati ottenuti, è stato scelto il modello XGBoost per effettuare la predizione del *trq_target* e, successivamente, calcolare il *trq_margin*.

2.4.2 Calcolo del Torque Margin

Dopo aver selezionato il modello XGBoost come migliore soluzione per la regressione, si è proceduto con il calcolo del *trq_margin*.
In primo luogo, sono stati predetti i valori del *trq_target* tramite il modello addestrato. Successivamente, è stata applicata la formula per il calcolo del *trq_margin* utilizzando i valori predetti e quelli misurati del torque.
Infine, il risultato ottenuto, denominato *trq_margin_pred*, è stato aggiunto al dataset, in modo da renderlo disponibile per la successiva fase di classificazione.

```
# Calcolo del torque margin utilizzando il torque target predetto
y_pred_trq_target = models["XGBoost"].predict(X_test_scaled)
trq_measured = merged_dataset.loc[X_test.index, 'trq_measured']

# Formula per il Torque Margin
trq_margin_pred = 100 * (trq_measured - y_pred_trq_target) / y_pred_trq_target

# Inserimento del torque margin predetto nel dataset
merged_dataset = merged_dataset.reindex(X_test.index)
merged_dataset['trq_margin_pred'] = trq_margin_pred
```

2.5 Probability Density Function

Dopo aver calcolato il *trq_margin*, il passo successivo consiste nella ricerca della Probability Density Function (PDF) che meglio descriva la distribuzione degli errori (residui) ottenuti dal modello di regressione.
Per farlo sono stati calcolati i residui del *trq_margin*, ovvero la differenza tra il valore reale e quello predetto.

```
# Calcoliamo i residui del torque margin
trq_margin_real = merged_dataset.loc[X_test.index, 'trq_margin']
residuals_trq_margin = trq_margin_real - trq_margin_pred
```

Successivamente, sono state adattate le distribuzioni teoriche (presentate in Sezione 2.1) ai residui, utilizzando le funzioni di fitting della libreria *scipy*. In questo modo si sono stimati i parametri ottimali per ciascuna distribuzione e si sono calcolate le relative PDF. Il codice seguente illustra questo procedimento:

```
# Funzione per il fit delle distribuzioni
def compare_pdf_fits(residuals, model_name):
    normal_params = norm.fit(residuals)
    laplace_params = laplace.fit(residuals)
    cauchy_params = cauchy.fit(residuals)
    logistic_params = logistic.fit(residuals)
    uniform_params = uniform.fit(residuals)

    x_vals = np.linspace(residuals.min(), residuals.max(), 300)

    normal_pdf = norm.pdf(x_vals, *normal_params)
    laplace_pdf = laplace.pdf(x_vals, *laplace_params)
    cauchy_pdf = cauchy.pdf(x_vals, *cauchy_params)
    logistic_pdf = logistic.pdf(x_vals, *logistic_params)
    uniform_pdf = uniform.pdf(x_vals, *uniform_params)
```

Attraverso il plot delle distribuzioni ottenute si riesce ad effettuare una valutazione qualitativa. Per effettuare, invece, una valutazione quantitativa, è stato utilizzato il KS Test già presentato nella sezione Sezione 2.1. Il test è stato applicato mediante la funzione *ks_1samp* della libreria *scipy*:

```
ks_results = {}
for name, (dist, params) in fitted_distributions.items():
    D, p_value = ks_1samp(residuals, lambda x: dist.cdf(x, *params))
    ks_results[name] = {"D-statistic": D, "p-value": p_value}
```

Di seguito, in Figura 2.3, è riportato il grafico che rappresenta le distribuzioni adattate.

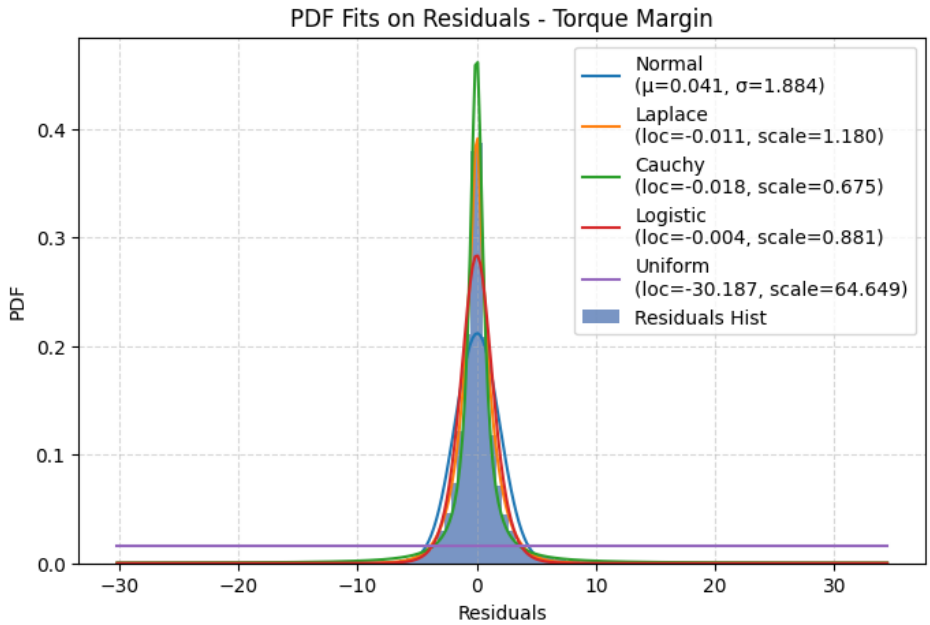

Figura 2.3: PDF fittate sui residui del *Torque Margin*

I risultati del KS Test, invece, sono riassunti nella Tabella 2.2.

Tabella 2.2: Risultati del test di Kolmogorov-Smirnov sulle distribuzioni fittate ai residui

Distribuzione	**D-statistic**	**p-value**
Normale	0.1146	0.0000
Laplace	0.0247	0.0000
Cauchy	0.0343	0.0000
Logistic	0.0608	0.0000
Uniforme	0.4448	0.0000

I risultati del test di Kolmogorov-Smirnov (KS) permettono di valutare quanto bene ciascuna distribuzione teorica si adatti ai residui ottenuti dal modello di regressione.

In tutti i casi il *p-value* risulta pari a 0.0000, il che implica che, da un punto di vista statistico, nessuna delle distribuzioni fittate riesce a rappresentare perfettamente i residui secondo il test formale. Tuttavia, confrontando i valori della statistica D, è possibile identificare quale distribuzione si avvicina maggiormente alla distribuzione effettiva dei residui.
La distribuzione di Laplace mostra il valore di D più basso (D = 0,0247), suggerendo il miglior adattamento tra quelle testate. Seguono la distribuzione Cauchy (D = 0,0343) e quella Logistica (D = 0,0608). La distribuzione Normale, comunemente utilizzata per modellare i residui, mostra invece un adattamento meno accurato (D = 0,1146). La distribuzione Uniforme, infine, risulta chiaramente inadatta (D = 0,4448), indicando un'elevata distanza dalla distribuzione reale dei residui.
Questi risultati indicano che la distribuzione di Laplace sia la scelta più adeguata per descriverne il comportamento.

Capitolo 3

Classificazione

Nel seguente capitolo viene esposta la procedura con la quale è stato possibile determinare lo stato di salute dei motori a turbina. Infatti, dopo aver determinato la stima dei valori del Torque Margin grazie alla strategia di regressione esposta nel Capitolo 2, possiamo impiegare tali valori, che sono stati inclusi all'interno del dataset di training (*trq_margin_pred*).

3.1 Selezione delle Feature

Le feature selezionate per eseguire il task di classificazione differiscono da quelle scelte per il precedente task di regressione. Questo è dovuto al fatto che la correlazione che deve essere tenuta ora in considerazione è quella tra le varie feature presenti nel dataset col parametro target del seguente task: *faulty*, indice dello stato di salute del motore (0 = sano, 1 = guasto).
A seguire sono riportate le feature scelte per eseguire la classificazione:

- **trq_margin_pred**: questa feature rappresenta il valore di *trq_margin* ottenuto dal precedente task di regressione e che si considera avere il medesimo valore di coefficente di correlazione col parametro target, ovvero pari a -0.61. Questo coefficente negativo è indice di una correlazione inversa tra le due feature, quando una aumenta, l'altra tende a dimunuire.

- **pa**: con un coefficente pari a 0.34, la feature Available Power risulta avere un grado di correlazione positivo che non è né eccessivamente basso, caso in cui la feature sarebbe stata esclusa; né troppo elevata. Difatti, un coefficente di

correlazione troppo elevato può favorire l'overfitting, andando a "copiare" troppo il comportamento del target.

- **trq_target**: presentando un coefficente di correlazione uguale a 0.31, tale feature è stata scelta per le medesime ragioni per cui è stata selezionata anche la feature precedemente riportata.

- **delta_T_relative**: anche nel caso di tale feature, con un coefficente di 0.35, il criterio di scelta è stato dalla logica precedentemente illustrata. Inoltre, avendo considerato la feature $\Delta T_{relative}$, è stato deciso di non comprendere tra le feature selezionate per la classificazione la Mean Gas Temperature e l'Outside Air Temperature. Questo perché quest'ultime due grandezze sono utilizzate per ricavare la feature derivata selezionata: selezionare sia la feature $\Delta T_{relative}$ che le feature da cui è derivata è risultato, in fase di sviluppo, non necessario e un'altra possibile causa di overfitting.

- **omega**: con una correlazione di 0.20, questa feature è stata selezionata poiché presenta un coefficente non eccessivamente alto, ma comunque meritevole di essere tenuto in considerazione nel corso dell'addestramento. Per di più tale feature, essendo anch'essa derivata, permette di tenere contemporaneamente in considerazione anche i contribuiti delle due feature da cui è derivata ($\omega = np/Torque_{measured}$). Tali feature, difatti, hanno dei coefficenti di correlazioni inferiori rispetto a quelli delle grandezze precedenti e perciò è stato ritenuto non necessario includerle nelle feature da fornire all'algoritmo di machine learning.

- **np_ng_ratio**: tale feature derivata ha un coefficente di correlazione di solo 0.12. Nonostante questo basso grado di correlazione, è stato deciso di tenerlo comunque in considerazione sulla base di osservazioni simili a quelle fatte per la precedente feature *omega*: una feature con coefficente basso, ma che permette di tenere in considerazione i contributi forniti dalle feature di base dalla quale è ricavata, ovvero Net Power e Compressor Speed.

Come è stato spiegato esaurientemente, l'esclusione delle feature non presenti nell'elenco appena fornito è stata, quindi, eseguita tenendo conto sia del basso grado di correlazione che presentavano, sia dell'eventuale contributo che hanno fornito nel calcolo delle feature derivate, risultando allora ridondante considerarle nuovamente nel corso dell'addestramento.

È qua riportata nuovamente la Correlation Heatmap di Figura 3.1 per maggiore comodità.

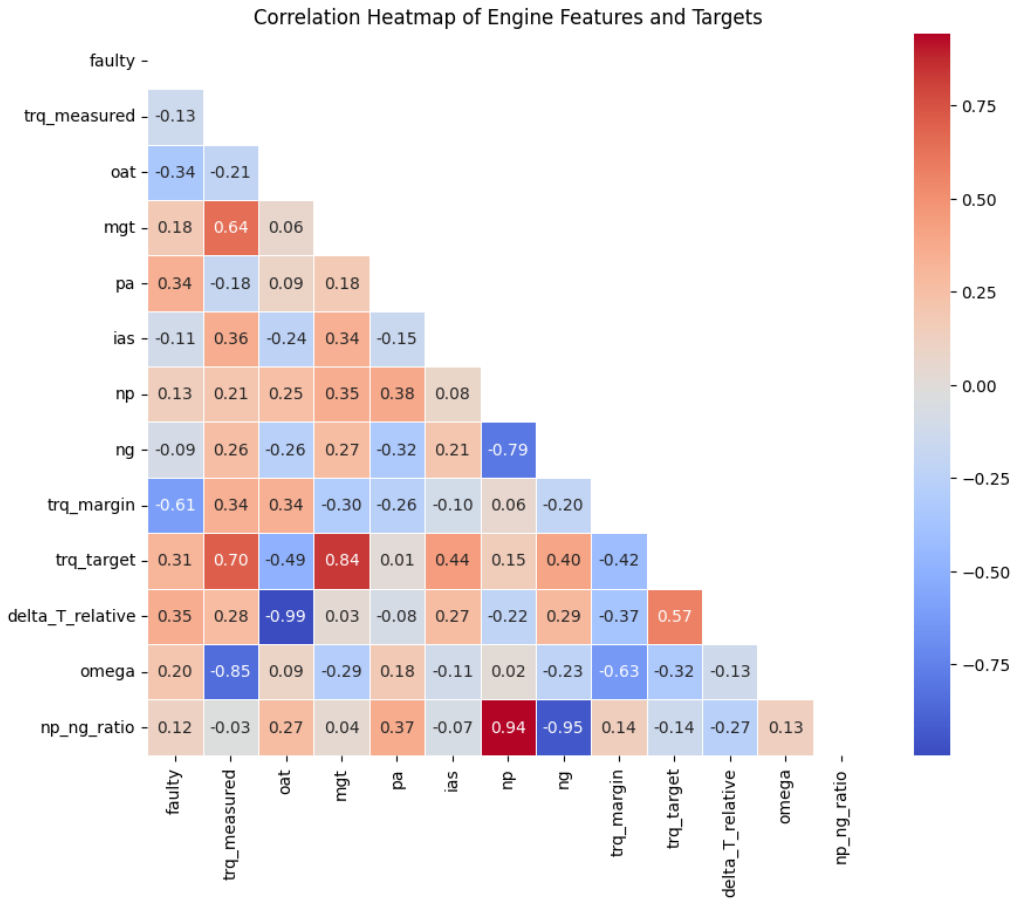

Figura 3.1: Correlation Heatmap delle Feature

3.2 Metodologia

Nel seguente paragrafo verranno illustrate le strategie che sono state impiegate per eseguire la classificazione degli elementi appartenenti al dataset.
Così come è stato effettuato per il task di regressione, è stata operata una suddivisione 80-20 del dataset in *training set* e *test set*, ovvero 80% del dataset di partenza è stato impiegato come dataset di training e il restante 20% come dataset di testing.
A seguito di questo è stata eseguita la normalizzazione, in modo da riscalare le feature, cosicché potessero avere una scala comune.

```
    # **Suddivisione in Training e Test Set**
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    stratify=y, random_state=42)

    # **Standardizzazione delle feature con dtype compatibile con GPU**
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train).astype(np.float32)
    X_test_scaled = scaler.transform(X_test).astype(np.float32)
```

Una volta preparato il dataset per l'addestramento, è stato necessario effettuare la scelta dei modelli di machine learning da impiegare per eseguire tale task. Anziché impiegare in modo tradizionale vari modelli classici di machine learning addestrandoli, ottenendo i risultati e confrontandoli per cercare quello che restituisse le migliori predizioni, la strategia scelta è stata quella dell'applicazione di un **Ensemble Model**, ossia un modello predittivo che combina più modelli di machine learning. Dal punto di vista pratico ciò che è stato fatto è consistito nell'addestramento di vari modelli di base, raccogliere i risultati ottenuti e usarli come input per l'addestramento di un ulteriore modello, detto *meta-modello*. Tutto ciò è stato fatto con lo scopo di ottenere una predizione complessiva più accurata, robusta e generalizzabile, riducendo gli errori individuali e la variabilità dei singoli modelli.

```
    # **Definizione dei modelli di base**
base_models = [
    ('RandomForest', RandomForestClassifier(n_estimators=50, max_depth=5,
    random_state=42)),
    ('AdaBoost', AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=5),
    n_estimators=50, random_state=42)),
    ('SVC', SVC(probability=True, kernel='rbf', C=1, random_state=42,
    cache_size=7000, verbose=True)),
    ('XGBoost', XGBClassifier(n_estimators=50, device='cuda' if use_gpu else 'hist',
                              max_depth=5, eval_metric='logloss', random_state=42))
]

# **Modello meta-classificatore: Logistic Regression**
meta_model = LogisticRegression(max_iter=500)

```

```
# **Creazione dello Stacking Classifier**
stacking_clf = StackingClassifier(estimators=base_models, final_estimator=meta_model,
cv=5)
```

Come riportato nel codice precedente, i modelli di base che sono stati scelti per andare a comporre l'Ensembe Model sono stati:

- **Random Forest**, modello di tipo *Bagging* robusto, che permette di ridurre la varianza.
- **AdaBoost**, modello di tipo *Boosting*, che permette di combinare modelli deboli in uno forte.
- **XGBoost**, anche questo di tipo *Boosting*, ma più ottimizzato grazie a tecniche di parallelizzazione, così da garantire migliori prestazioni sia dal punto di vista dell'efficacia che dell'efficienza.
- **SVC**, di tipo *Margin-Based*, quindi di tipo diverso dai precedenti, in grado di gestire separazioni non lineari e spazi ad alta dimensionalità.
- **Logistic Regression**, utilizzato come *meta-modello* data la sua leggerezza, velocità e capacità di generalizzare.

L'utilizzo di un insieme variegato di modelli come quello appena definito, all'interno dello stacking ensemble, permette di ottenere buone prestazione e ottima robustezza, oltre che un buon grado di generalizzazione.
La scelta di modelli di base di diverso tipo è ottimale in quanto modelli di tipo diverso tenderanno a commettere errori di tipo diverso e, combinando le loro previsioni, aumenterà la probabilità che almeno uno dei modelli catturi i pattern corretti. I modelli di base individuati permettono inoltre di ridurre varianza e bias, rimanendo efficaci in spazi complessi, consentendo, quindi, di avere una migliore capacità predittiva. Tuttavia, un difetto di questo tipo di modelli è il tempo di training, che è risultato essere di gran lunga maggiore rispetto a quello che sarebbe stato necessario per l'addestramento dei singoli modelli.

3.3 Addestramento dei Modelli

Ora che è stato descritto l'approccio adottato per la procedura di classificazione si procederà ad illustrare i passaggi seguiti dal punto di vista più implementativo e tecnico per arrivare ai risultati finali.
Da non trascurare è la scelta dei parametri che è stata adottata per poter ottimizzare al meglio le performance dei classificatori utilizzati. A seguire sono riportate i parametri principali di ogni modello che sono visibili nel codice 3.2:

- **Random Forest**:
  - *n_estimator = 50*, numero di alberi della foresta. Questo valore del parametro *n_estimator* è stato impostato allo stesso modo anche negli altri modelli che lo richiedevano, in modo da avere dei risultati coerenti tra i vari modelli. Il valore è stato scelto pari a 50 in modo da avere una discreta stabilità, una buona velocità di elaborazione ed evitare di incorrere in un possibile overfitting. Un valore troppo alto di questo parametro, infatti, può aumentare il rischio di questa evenienza.
  - *max_depth = 5*, profondità massima di ogni albero. Anche questo parametro è stato impostato nello stesso modo per gli altri modelli. È stato scelto questo valore per evitare overfitting e per accelerare i tempi di elaborazione del modello.
- **AdaBoost**:
  - *estimator=DecisionTreeClassifier(max_depth = 5)*, serve a specificare il modello debole su cui il modello AdaBoost si basa, nel caso specifico alberi con profondità massima pari a 5.
  - *n_estimators = 50*, come precedentemente descritto, questo parametro è settato uguale per tutti.
- **SVC**:
  - *probability = True*, parametro che, impostato in questo modo, va a calcolare le probabilità delle classi, necessarie per lo stacking ensemble.
  - *kernel = 'rbf'*, utile per selezionare il tipo di kernel impiegato, in questo caso *Radial Basis Function*.

  - *C = 1*, parametro di regolarizzazione con cui si controlla quanto penalizzare gli errori di classificazione. Un valore pari a 1 è un valore intermedio che bilancia le necessità di accuratezza del training set con quelle di generalizzazione del test set.
  - *cache_size = 7000*, rappresenta la quantità di memoria (in MB) allocata per il kernel. Questo è un parametro utile soprattutto quando si devono analizzare dataset grandi come quello elaborato nel corso di questa trattazione.
  - *verbose = True*, utile a stampare l'output del processo di training.

- **XGBoost**:
  - *n_estimators = 50*, parametro impostato in tal modo per le ragioni precedentemente illustrate
  - *device = 'cuda' if use_gpu else 'hist'*, parametro utilizzato per far sfruttare al modello la GPU, qualora fosse disponibile e utilizzabile, permettendo di accelerare i tempi di elaborazione del modello.
  - *max_depth = 5*, parametro impostato nel medesimo modo degli altri modelli.
  - *eval_metric = 'logloss'*, con cui viene definita la funzione di perdita usata per la valutazione.

- **Logistic Regression**:
  - *max_iter = 500*, ossia il numero massimo di iterazioni per la convergenza dell'ottimizzazione. Il valore di default è pari a 100, ma è stato aumentato date le grandi dimensioni del dataset in esame.

Per tutti questi modelli è presente, ovviamente, anche il parametro *random_state = 42*, utile a rendere l'output degli addestramenti dei vari algoritmi deterministico e riproducibile.
Per eseguire la classificazione sono stati adottati due approcci e i risultati ottenuti sono stati confrontati per vedere quale dei due restituisse i risultati più soddisfacenti: *classificazione uniforme* e *classificazione con pesi*.
La classificazione uniforme è stata operata eseguendo il codice sopra riportato ed eseguendo l'effettivo addestramento con la seguente istruzione:

```
1    stacking_clf.fit(X_train_scaled, y_train)
```

Classificazione con Pesi La classificazione iniziale è stata eseguita trattando in modo uniforme entrambe le classi, Nominal (classe 0) e Faulty (classe 1). Tuttavia, in questo contesto applicativo, gli errori nella previsione della classe Faulty risultano particolarmente critici, poiché un falso negativo (ovvero classificare come Nominal un motore guasto) può avere conseguenze operative rilevanti.
Per questo motivo, è stata adottata una classificazione pesata, con l'obiettivo di dare maggiore importanza alla classe Faulty durante l'addestramento. Questo approccio consente di penalizzare maggiormente gli errori su tale classe e migliorare la capacità del modello nel rilevare condizioni di guasto, privilegiando la riduzione dei falsi negativi rispetto alla massimizzazione dell'accuratezza complessiva.
A seguire è riportato il codice con cui sono stati impostati i pesi tra i parametri dei vari modelli impiegati. Per tutti i modelli è stata impostato un parametro *class_weight* = {*0:1, 1:3*} oppure *scale_pos_weight* = *3*, con cui si va penalizzare gli errori sulla classe 1, cosicchè ogni errore sulla classe 0 conti 1, mentre ogni errore sulla classe 1 contino 3.

```
    # Definizione dei modelli di base con class weights
base_models = [
    ('RandomForest', RandomForestClassifier(
        n_estimators=50, max_depth=5, random_state=42,
        class_weight={0:1, 1:3}
    )),
    ('AdaBoost', AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=5, class_weight={0:1, 1:3}),
        n_estimators=50, random_state=42
    )),
    ('SVC', SVC(
        probability=True, kernel='rbf', C=1, random_state=42,
        cache_size=7000, verbose=True,
        class_weight={0:1, 1:3}
    )),
    ('XGBoost', XGBClassifier(
        n_estimators=50,
        device='cuda' if use_gpu else 'cpu',  # Usa GPU se disponibile
        max_depth=5, eval_metric='logloss', random_state=42,
        scale_pos_weight=3
    ))
```

```
]

# Modello meta-classificatore con class weight
meta_model = LogisticRegression(
    max_iter=500,
    class_weight={0:1, 1:3}
)
```

3.4 Risultati della Classificazione

Una volta terminato l'addestramento sono stati raccolti ed esaminati i risultati della classificazione.

```
# Predizione delle etichette
y_pred = stacking_clf.predict(X_test_scaled)

# Calcolo delle probabilità di ciascuna classe
y_proba = stacking_clf.predict_proba(X_test_scaled)

# Calcolo del confidence score
confidence_scores = []
for i in range(len(y_pred)):
    confidence_scores.append(y_proba[i, y_pred[i]])
```

Col codice riportato precedentemente sono stati calcolati i *confidence score* per ogni predizione, ovvero la probabilità che il modello assegna alla classe che ha predetto.
A seguire è stata calcolata l'*accuracy* del modello, restituita come la percentuale di predizioni corrette, ed è stato stampato il *classification report*, contenente importanti metriche di valutazione del modello, quali *precision*, *recall* ed *f1-score* per ogni classe. Inoltre, è stata generata anche la matrice di confusione relativa ai risultati ottenuti.

```
# Calcolo delle metriche
accuracy = accuracy_score(y_test, y_pred)
print(f"\n Accuracy (modello caricato): {accuracy:.4f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred, digits=4))
```

```python

# Matrice di confusione
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="coolwarm",
            xticklabels=["Healthy", "Faulty"], yticklabels=["Healthy", "Faulty"])
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix - Loaded Model")
plt.show()
```

Fatto ciò, è stato creato un nuovo DataFrame contenente i *confidence score* delle predizioni effettuate ed è stata eseguita un'analisi dei falsi negativi, andando a identificare all'interno del DataFrame tutti i falsi negativi e, poi, selezionando tutti i falsi negativi (FN) con un *confidence score* di cui il modello era molto sicuro ($> 70\%$).

```python
# Creazione DataFrame con confidence score
df_results = pd.DataFrame({
    'TrueLabel': y_test.values,
    'PredictedLabel': y_pred,
    'ConfidenceScore': confidence_scores
})
# Identifica i falsi negativi (FN)
#    FN: TrueLabel=1 (Faulty), PredictedLabel=0 (Healthy)
df_fn = df_results[
    (df_results['TrueLabel'] == 1) &
    (df_results['PredictedLabel'] == 0)
].copy()

# Seleziona quelli con ConfidenceScore > 0.7
df_fn_high_conf = df_fn[df_fn['ConfidenceScore'] > 0.7]

n_fn_total = len(df_fn)
n_fn_high_conf = len(df_fn_high_conf)

print(f"Falsi negativi totali: {n_fn_total}")
```

```
print(f"Falsi negativi con confidenza > 0.7: {n_fn_high_conf} "
      f"({n_fn_high_conf/n_fn_total*100:.2f}% dei FN)")
```

Così facendo si è riusciti a valutare quanti falsi negativi "sicuri" sono stati predetti dai due modelli, così da poter selezionare il migliore, ma anche per poter, in generale, valutare le performance dello stacking ensemble progettato. Un'attenzione maggiore è stata posta sui falsi negativi sicuri poiché, in un ambito applicativo come quello in esame, questi sono gli errori più critici e pericolosi che potrebbero portare a trascurare un eventuale guasto al motore dell'elicottero.

3.4.1 Scoring

Un ulteriore metodo utilizzato per valutare i risultati ottenuti dall'addestramento dei modelli è stata la funzione di scoring fornita dai creatori della challenge PHM America 2024 per poter eseguire la valutazione dei risultati.

```
def custom_score_fn(true_label, pred_label, confidence):
    if (confidence < 0) or (confidence > 1):
        return -100

    if pred_label not in [0, 1]:
        return -100

    if pred_label != true_label:
        confidence = -confidence

    if true_label == 0:
        score = confidence
    else:
        if confidence >= 0:
            score = confidence
        else:
            score = 4 * (confidence ** 11) + 1.0 * confidence

    return score


```

```
df_results_with_score = df_results.copy()
df_results_with_score['Score'] = df_results_with_score.apply(
    lambda row: custom_score_fn(row['TrueLabel'],
    row['PredictedLabel'], row['ConfidenceScore']),
    axis=1
)

final_score = df_results_with_score['Score'].mean()
print(f"Score finale medio sulla classificazione: {final_score:.4f}")

total_score = df_results_with_score['Score'].sum()
print(f"Score totale sulla classificazione: {total_score:.4f}")
```

Grazie alla funzione messa a disposizione dagli organizzatori della challenge è stato possibile eseguire un'ulteriore valutazione dei risultati ottenuti che verranno presentati nel paragrafo seguente.
Questa funzione di *scoring*, genera una valutazione per la classe *faulty* rappresentata dalla curva in Figura 3.2. Da questo grafico è stato scelto il valore soglia 0.7 per valutare la distribuzione dei falsi negativi, perchè a livello visivo è il valore in cui la curva inizia a descrescere in modo più netto.

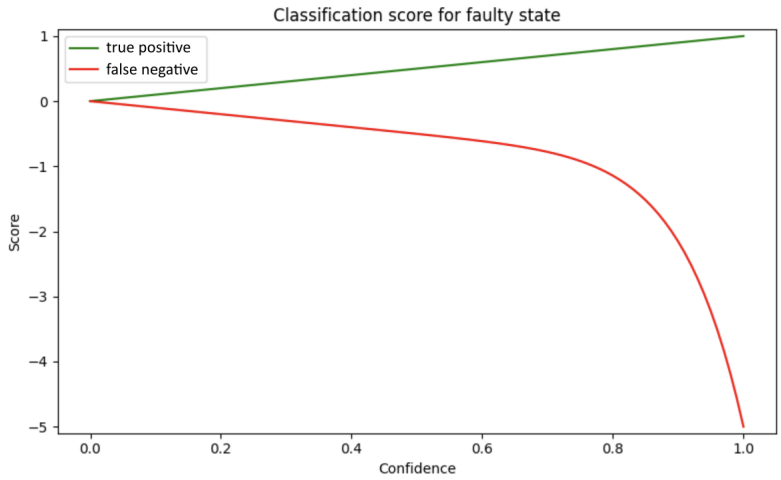

Figura 3.2: Grafico della funzione di Score della challenge

3.4.2 Performance dei Modelli

A seguire sono mostrati i risultati ottenuti in seguito ai due addestramenti precedentemente illustrati.

Performance della Classificazione con Pesi Il seguente è il *classification report* ottenuto dall'addestramento con pesi, Figura 3.3.

```
 Accuracy: 0.9773

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.96      0.98     17674
           1       0.96      0.99      0.98     17783

    accuracy                           0.98     35457
   macro avg       0.98      0.98      0.98     35457
weighted avg       0.98      0.98      0.98     35457


Falsi negativi totali: 158
Falsi negativi con confidenza > 0.7: 107
67.72% dei FN

Score medio: 0.9416
Score totale: 33386.6654
```

Figura 3.3: Classification Report relativo alla Classificazione Pesata

La metrica più importante per valutare l'efficacia dell'addestramento operato è, sicuramente, l'*accuracy* che, attestandosi al valore di 97.73%, risulta avere un valore molto elevato, suggerendo, quindi, che la classificazione è stata eseguita in modo molto accurato.
Da sola, però, l'*accuracy* non è in grado di fornire un quadro completo per quanto riguarda la valutazione del problema di classificazione in questione. Ad affiancarla ci sono altre metriche quali la *precision*, la *recall* e l'*F-1 Score*: questi tre indicatori hanno tutti dei valori molto elevati. Nel caso particolare della classificazione pesata, si può osservare come il modello sia leggermente migliore nel classificare le istanze *faulty*, registrando un valore della *recall* per questa classe pari a 0.99. Questo indicatore, nella classe 0, nonostante abbia comunque un valore molto elevato, risulta essere leggermente inferiore, ovvero di 0.96. Per entrambe le classi, il valore dell'*F1-Score* è uguale a 0.98, suggerendo un ottimo equilibro tra *precision* e *recall* calcolati per le due classi.
A seguire è stato registrato anche il numero di falsi negativi totali che sono stati ricavati dall'addestramento, pari a 158, e di questi oltre 2/3 avevano una confidenza superiore

al valore di 0.7. Tale valore soglia è un valore di confidenza che è stato fissato per discriminare quantitativamente le istanze che il modello ha erroneamente classificato come *normale* e di cui era molto sicuro. Ciò è indice che ci sono diverse istanze del dataset che risultano essere complesse da distinguere per il modello.

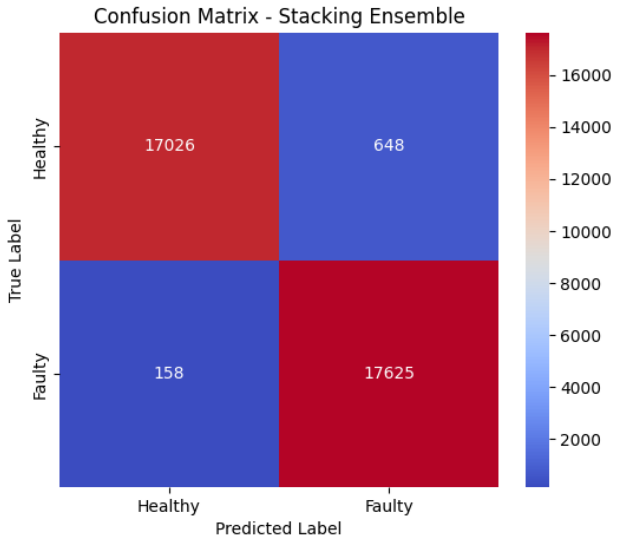

Figura 3.4: Matrice di Confusione relativa alla Classificazione Pesata

Da tenere in considerazione, tuttavia, è anche il fatto che, in rapporto al numero di istanze che sono state correttamente classificate, il numero di falsi positivi e negativi è nettamente inferiore. Questa osservazione è supportata dalla matrice di confusione mostrata a Fig. 3.4.
Ultimo dato riportato all'interno del *classification report* è lo score medio e totale ottenuto grazie alla funzione di scoring messa a disposizione dai creatori della challenge e riportata nel Parag. 3.4.1. Lo *score medio*, pari a 0.9416, ci fornisce un'indicazione della qualità media delle predizioni, andando a bilanciare precisione e confidenza. Lo *score totale*, uguale a 33386.6654, ci dà un'idea della performance complessiva del modello su tutti i dati, tenendo conto delle predizioni corrette e degli errori.

Performance della Classificazione Senza Pesi Qui di seguito mostriamo il *classification report* prodotto dalla classificazione eseguita senza l'applicazione dei pesi sulle classi, Figura 3.5
Si può riscontrare come l'*accuracy* ottenuta dopo questo secondo tipo di addestramento sia leggermente più elevata rispetto a quella precedente, attestandosi ad un 98.54%.

```
 Accuracy: 0.9854

Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.99      0.99     17674
           1       0.99      0.98      0.99     17783

    accuracy                           0.99     35457
   macro avg       0.99      0.99      0.99     35457
weighted avg       0.99      0.99      0.99     35457


Falsi negativi totali: 276
Falsi negativi con confidenza > 0.7: 176
63.77% dei FN

Score medio: 0.9568
Score totale: 33924.7883
```

Figura 3.5: Classification Report relativo alla Classificazione con Pesi Uniformi

Questa prima metrica è, quindi, già indicativa di come questo secondo tipo di classificazione riesca a restituire risultati migliori. Proseguendo con l'analisi del *classification report* si osserva come anche i parametri relativi a *precision*, *recall* e *F1-score* risultino essere eccellenti, con valori per tutte le metriche attorno allo 0.98/0.99 e ciò riscontrabile per entrambe le classi. Dall'analisi dei falsi negativi operata si può evincere come, nonostante siano stati individuati un numero più elevato di quest'ultimi, pari a 276, la percentuale di FN con confidenza superiore alla soglia fissata più 0.7 risulta essere inferiore a quella ottenuta dalla classificazione con pesi, attestandosi ad un 63.77%. Difatti il numero di falsi negativi con un valore di confidenza superiore alla soglia impostata è di 176. Questo è indice di come il classificatore sia lievemente più "insicuro" quando va a distinguere istanze di questo tipo. Anche i valori di scoring ottenuti tramite la funzione di valutazione resa disponibile risultano essere migliori di quelli ottenuti precedentemente: con uno *score medio* di 0.9568 e uno *score totale* pari a 33924.7883 entrambi i valori rispecchiano delle performance complessive molto buone, sicure e tendenzialmente corrette da parte del classificatore. Per operare la scelta su quale classificatore sia il più adatto al task in esame, è opportuno valutare sia l'*accuracy* complessiva del modello, sia le performance relative all'individuazione dei falsi negativi: se si ritiene necessario dare una maggiore importanza al numero di falsi negativi, volendo avere il minor numero possibile di quest'ultimi, allora si può optare per il classificatore pesato, che risulta performare meglio rispetto alla sua controparte con pesi uniformi. Ai fini

della challange in esame è stato ritenuta una scelta migliore selezionare il classificatore con pesi uniformi, dato che complessivamente le performance che ha ottenuto sono state più soddisfacenti.

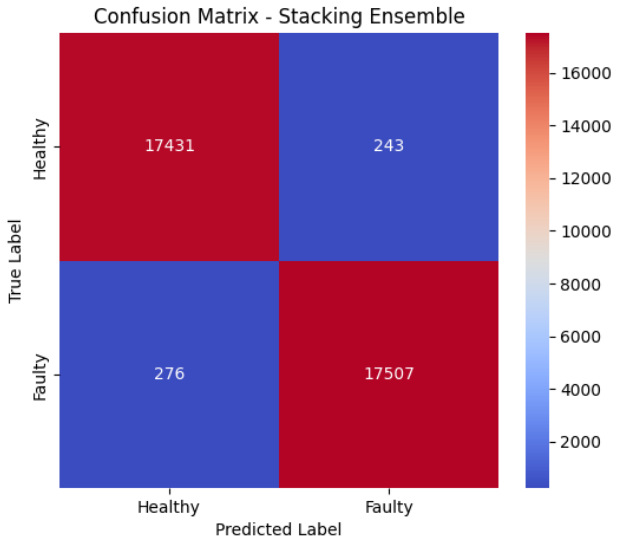

Figura 3.6: Matrice di Confusione relativa alla Classificazione con Pesi Uniformi

I dati relativi alla distibuzione di istanze classificate correttamente o meno sono visibile anche dalla matrice di confusione riportata in Figura 3.6.

Conclusioni

Nella trattazione appena esposta, è stata affrontata la *PHM North America 2024 Conference Data Challenge*, avente come obiettivo lo stimare lo stato di salute di un motore a turbina di un elicottero. Per far ciò sono state illustrate le tecniche di regressione impiegate, che hanno consentito di stimare i valori della feature Torque Margin, ovvero la differenzia tra la coppia desiderata e quella fornita dal motore. Una volta eseguito tutto questo, sono state eseguite due tipologie di classificazione differenti per assicurarci di ottenere un risultato quanto il più soddisfacente possibile. Per questa fase è stato implementato un *ensemble model*, ossia un metodo di apprendimento automatico che combina le previsioni ricavate dall'addestramento di singoli modelli. Entrambe queste fasi sono state, ovviamente, precedute da una fase di pre-processing del dataset, poco rilevante dato che il dataset fornito era già predisposto per poter essere impiegato nei task di regressione e classificazione, e una fondamentale fase di selezione delle feature, dove sono state individuate gli attributi del dataset più rilevanti e con più contenuto informativo. Questa fase è stata di grande importanza anche ai fini di ottimizzazione dei tempi di addestramento dei modelli di machine learning. Con l'implementazione della pipeline precedentemente descritta, la challenge è stata completata in modo efficente, fornendo dei risultati dalla fase di classificazione che riteniamo essere di ottima qualità.